Indexing and Slicing with Pandas*

Samuel Dayo Adesola

March 22, 2026

1 Rows with `.loc`

Instead of memorizing the integer positions to locate the order, age, height, and party information of Abraham Lincoln, with DataFrame, we can access it by name using `.loc`:

```
import pandas as pd

presidents_df = pd.read_csv(
    'president_heights_party.csv',
    index_col='name'
)

print(presidents_df.loc['Abraham Lincoln'])
```

The result is a pandas Series of shape (4,).

```
print(type(presidents_df.loc['Abraham Lincoln']))
print(presidents_df.loc['Abraham Lincoln'].shape)
```

We can also slice by index. Say we are interested in gathering information on all of the presidents between Abraham Lincoln and Ulysses S. Grant:

```
print(presidents_df.loc['Abraham Lincoln':'Ulysses S. Grant'])
```

The result is a new DataFrame, a subset of `presidents_df`. `.loc[]` allows us to select data by label or by a conditional statement.

2 Rows with `.iloc`

Alternatively, if we know the integer position(s), we can use `.iloc` to access the row(s).

```
print(presidents_df.iloc[15])
```

To gather information from the 16th to 18th presidents:

```
print(presidents_df.iloc[15:18])
```

Both `.loc[]` and `.iloc[]` may be used with a boolean array to subset the data.

3 Columns

We can retrieve an entire column from `presidents_df` by name. First, we access all column names:

*Adapted from Sololearn

```
print(presidents_df.columns)
```

This returns an index object containing all column names. Then we can access the column `height`:

```
print(presidents_df['height'])
```

```
print(presidents_df['height'].shape)
```

This returns a Series containing heights from all U.S. presidents.

To select multiple columns, we pass the names in a list, resulting in a DataFrame:

```
print(presidents_df[['height', 'age']].head(n=3))
```

In this way, we focus only on the columns of interest.

When accessing a single column, one bracket returns a Series (single dimension), while double brackets return a DataFrame (multi-dimensional).

4 More with `.loc`

If we want to access columns `order`, `age`, and `height`, we can do so with `.loc`. For example, to access columns from `order` through `height` for the first three presidents:

```
print(presidents_df.loc[:, 'order':'height'].head(n=3))
```

The index in pandas makes retrieving information from rows or columns convenient and efficient, especially when the dataset is large or contains many columns. Therefore, we do not need to memorize the integer positions of each row or column.